

◆ Gemini

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import Insert code cell options

# --- Step 1: Download a high-resolution image to work with ---
image_url = 'https://cdn.pixabay.com/photo/2016/11/29/05/45/car-1867803_1280.jpg' # A reliable image URL
# Using a generic test image URL that is usually reliable
image_url = 'https://i.imgur.com/Kx20YnQ.jpg' # Example image from Imgur
image_path = '/content/autonomous_car_scene.jpg'

original_image = None # Initialize to None

try:
    response = requests.get(image_url)
    response.raise_for_status() # Added a timeout
    response.raise_for_status() # Raise an exception for HTTP errors
    with open(image_path, 'wb') as handler:
        handler.write(response.content)
    print(f"Image downloaded successfully to {image_path}")

    # Try to load the downloaded image
    original_image = cv2.imread(image_path)
    if original_image is None:
        print(f"Error: Could not load image from {image_path} after download. It might be corrupted or an invalid image file.")
    else:
        original_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)
        print(f"Original image shape: {original_image.shape}")

except requests.exceptions.RequestException as e:
    print(f"Error downloading image: {e}")
    image_path = None

if image_path:
    original_image = cv2.imread(image_path)
    original_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)
    print(f"Original image shape: {original_image.shape}")
else:
    print("Cannot proceed without a downloaded image.")
    print("Attempting to generate a synthetic image as a fallback...")

# Fallback: If original_image is still None (either download failed or failed to load)
if original_image is None:
    print("Generating a simple synthetic image instead.")
    # Create a simple checkerboard pattern or gradient as a fallback
    height, width = 480, 640 # Standard dimensions
    original_image = np.zeros((height, width, 3), dtype=np.uint8)
    # Create a simple gradient
    for i in range(height):
        original_image[i, :, 0] = int(255 * (i / height)) # Red channel gradient
        original_image[i, :, 1] = int(255 * (1 - i / height)) # Green channel gradient
    original_image[:, width // 2:, 2] = 200 # Blue half
    original_image[:height // 2, :, 0] = 255 # Red top half
    print(f"Synthetic image generated with shape: {original_image.shape}")

# Define height and width variables globally for subsequent cells
height, width, _ = original_image.shape if original_image is not None else (0, 0, 0)
```

Error downloading image: 429 Client Error: Unknown Error for url: <https://i.imgur.com/Kx20YnQ.jpg>
 Attempting to generate a synthetic image as a fallback...
 Generating a simple synthetic image instead.
 Synthetic image generated with shape: (480, 640, 3)

```
# --- Step 2: Simulate Degradation (Low Resolution and Noise) ---
if 'original_image' in locals() and original_image is not None:
    # 2a. Simulate Low Resolution (Downsampling)
    # Reduce the image size by a factor, then upscale using simple interpolation
    downscale_factor = 4
    height, width, _ = original_image.shape
    low_res_height = height // downscale_factor
    low_res_width = width // downscale_factor

    low_res_img = cv2.resize(original_image, (low_res_width, low_res_height), interpolation=cv2.INTER_AREA)
```

```
# 2b. Add Gaussian Noise to the low-resolution image
noise_mean = 0
noise_sigma = 25 # Standard deviation of the noise
gauss = np.random.normal(noise_mean, noise_sigma, low_res_img.shape).astype('uint8')
noisy_low_res_img = cv2.add(low_res_img, gauss) # Add noise
    Insert code cell options
print(f"Low-resolution noisy image shape: {noisy_low_res_img.shape}")
else:
    print("Original image not available for degradation simulation.")
```

Low-resolution noisy image shape: (120, 160, 3)

```
# --- Step 3: Apply Enhancement (Upscaling and Denoising) ---
if 'noisy_low_res_img' in locals() and noisy_low_res_img is not None:
    # 3a. Super-Resolution (Upscale the image)
    # Use bicubic interpolation for a better quality upscaling than nearest-neighbor
    upscaled_img = cv2.resize(noisy_low_res_img, (width, height), interpolation=cv2.INTER_CUBIC)

    # 3b. Denoising
    # Apply a Non-Local Means Denoising filter for effective noise reduction while preserving edges
    # Parameters: src, h, hColor, templateWindowSize, searchWindowSize
    # h: parameter for noise level. A larger h value removes more noise but can blur image features.
    # hColor: same as h, but for color components. Usually set to h or slightly less.
    # templateWindowSize: size of the pixel neighborhood used to compute the weighted average. Usually 7.
    # searchWindowSize: size of the window to search for similar patches. Usually 21.
    enhanced_img = cv2.fastNlMeansDenoisingColored(upscaled_img, None, 10, 10, 7, 21)

    print(f"Enhanced image shape: {enhanced_img.shape}")
else:
    print("No degraded image available for enhancement.")
```

Enhanced image shape: (480, 640, 3)

```
# --- Step 4: Display Results ---
if 'original_image' in locals() and original_image is not None and \
'noisy_low_res_img' in locals() and noisy_low_res_img is not None and \
'enhanced_img' in locals() and enhanced_img is not None:

    plt.figure(figsize=(18, 6))

    plt.subplot(1, 3, 1)
    plt.imshow(original_image)
    plt.title('Original High-Resolution Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    # Upscale the noisy_low_res_img for display comparison, using the same interpolation
    display_degraded_img = cv2.resize(noisy_low_res_img, (width, height), interpolation=cv2.INTER_NEAREST) # Use nearest for sh
    plt.imshow(display_degraded_img)
    plt.title('Simulated Degraded Image (Low-Res + Noise)')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(enhanced_img)
    plt.title('Enhanced Image (Upscaled + Denoised)')
    plt.axis('off')

    plt.tight_layout()
    plt.show()
else:
    print("Cannot display results due to missing images.")
```

